

Reproducible Computing in R with {rix} and Nix

Felipe Fontana Vieira

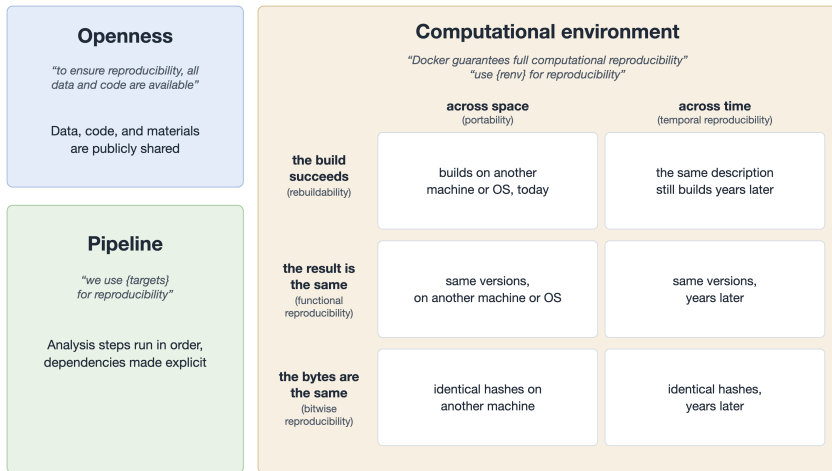
Ghent University

Based on joint work with Jason Geller (Boston College)
and Bruno Rodrigues (Ministry of Research and Higher Education, Luxembourg)

Disclaimers before moving on

1. My ongoing PhD is not on reproducibility.
2. I am not officially connected to Nix and `{rix}`. I have done *minor* contributions to `{rix}`, but nothing that creates a conflict of interest.

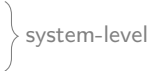
How we talk about reproducibility in the wild



(Baker et al., 2024; Epskamp, 2019; Lamb & Zacchiroli, 2022; Landau, 2021; Levenstein & Lyle, 2018; Luijken et al., 2024; Malka, Zacchiroli, & Zimmermann, 2024, 2025, 2026; Nosek et al., 2015; Peikert & Brandmaier, 2021; Peikert, van Lissa, & Brandmaier, 2021; Rodrigues, 2023; Rodrigues & Baumann, 2026; Siepe et al., 2024; Ziemann, Poulain, & Bora, 2023)

The computational environment

Setting bitwise reproducibility aside, even error-free R code sits on top of four layers, each of which may drift over space and time:

1. **R version**
 2. **R packages**
 3. **Document tools:** e.g., Pandoc, LaTeX, Quarto
 4. **System libraries and compilers:** e.g., gcc, libpng, BLAS/LAPACK
- 

Some examples based on my personal experience, but that are general:

1. R version

- ▶ **Syntax:** the native pipe `|>` exists only since R 4.1.0.
- ▶ **Defaults:** R 4.0.0 stopped turning strings into factors. Same code, different object:

```
df <- data.frame(group = c("a", "b")); class(df$group); as.numeric(df$group)
#> R 3.6.3:  "factor"      1  2      (stringsAsFactors = TRUE)
#> R 4.0.0+: "character" NA NA      (stringsAsFactors = FALSE)
```

- ▶ **Random numbers:** R 3.6.0 changed how `sample()` draws. Same seed, different draws:

```
set.seed(123); sample(1:10, 3)
#> R 3.5.3:    3  8  4      (sample.kind = "Rounding")
#> R 3.6.0+:    3 10  2      (sample.kind = "Rejection")
```

Note: "Rounding" is R's retrospective name for the pre-3.6.0 behaviour; `sample.kind` only exists since 3.6.0. (Fontana Vieira, Geller, & Rodrigues, 2026; Ottoboni & Stark, 2018)

2. R packages

- ▶ **Renamed or deprecated:** `dplyr::cur_data()`, but now `dplyr::pick()` (warning about the future).
- ▶ **Internal functions** (discouraged, but used nonetheless):

```
lavaan:::lav_utils_get_ancestors(...)  
#> lavaan 0.6-20: works  
#> lavaan 0.6-21+: Error: object 'lav_utils_get_ancestors' not found  
#> (now lavaan:::lav_graph_get_ancestors)
```

2. R packages (continued)

- **Defaults:** Nothing errors, nothing warns, the target covariance, skewness and kurtosis are all still met. Same seed, different data:

```
set.seed(123); sigma.target <- matrix(c(1, .5, .5, 1), 2)
dat <- covsim::rIG(1000, sigma.target, skew = c(1, 1), excesskurt = c(2, 2))
dat[[1]][1, ]
#> covsim 1.0.0:  -0.86  0.34      (typeA = "symm")
#> covsim 1.1.0:   0.29  0.99      (typeA = "triang")
```

3. Document tools

- ▶ Pandoc and LaTeX, called by Quarto (also outside R) or by `{rmarkdown}`. Each has its own version.
- ▶ From this talk's demo: HTML fine, PDF failed until one more LaTeX package was declared:

LaTeX Error: File ``lualatex-math.sty'` not found.

```
rix::rix(..., tex_pkgs = c("amsmath", ..., "lualatex-math"))
```

(Rodrigues & Baumann, 2026)

4. System libraries and compilers

- ▶ Many R packages are compiled code (C, C++, Fortran) and use libraries from the OS (e.g., libpng, BLAS/LAPACK).
- ▶ Today: CRAN hands you a binary. However, years later: only the source. Your machine compiles it, with today's compiler and today's libraries.
- ▶ Same script, different BLAS (illustration in next slides).
- ▶ **With Nix** (to be explained): the compiler and the libraries are part of the recipe. Prebuilt per OS for every date, rebuildable if not.

(Fontana Vieira, Geller, & Rodrigues, 2026; Navarro, 2025)

4. System libraries and compilers (continued)

- ▶ Based on an illustration by Danielle Navarro; eigenvector sign flip in the system's BLAS/LAPACK (`MASS::mvrnorm()` is not invariant, `mvtnorm::rmvnorm()` fine).

```
== MASS (native R) ==
```

```
macOS:      0.28780065222640439    0.59425558497432929   -0.25961849316812602   -2.3918999759288693
Ubuntu:    -0.43918332712576574    0.25608933342594919    0.85420518537273971   -2.2883238259260961
Windows:  -0.43918332712576580    0.25608933342594914    0.85420518537273971   -2.2883238259260965
=> DIVERGE
```

- ▶ With identical code, inside a Nix environment:

```
== MASS (Nix) ==
```

```
macOS:      -0.43918332712576924    0.25608933342595080    0.85420518537274215   -2.2883238259260961
Ubuntu:    -0.43918332712576469    0.25608933342594986    0.85420518537273837   -2.2883238259260970
=> MATCH (to 10 significant digits)
```

- ▶ However, there are cases where Nix will simply not solve the difference (e.g., floating-point arithmetic, number of threads across CPUs, hardware etc, may play a relevant role in differences).

Managing the environment

- ▶ Issue: combining incomplete tools (i.e., stabilize parts of it), which may require diverse expertise (e.g., `{renv}`, `{groundhog}`, `{rang}`, `{rig}`, and Docker).
- ▶ More complicated with other programming languages (e.g., Julia and Python).
- ▶ Recommendation: *Docker does not guarantee reproducibility* (Malka, Zacchioli, & Zimmermann, 2026).

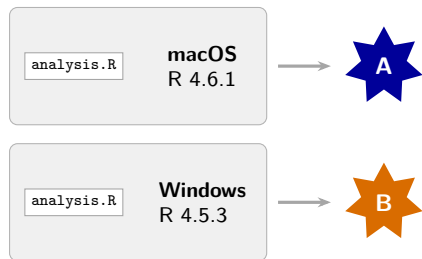
(Chan & Schoch, 2023; Csárdi, 2026; Fontana Vieira, Geller, & Rodrigues, 2026; Ottoboni & Stark, 2018; Peikert & Brandmaier, 2021; Rodrigues & Baumann, 2026; Simonsohn & Gruson, 2026; Ushey & Wickham, 2024; Wiebels & Moreau, 2021)

Disclaimer: {wvr} and ir are relevant, but they literally just became available (summer 2026). Also, the Posit Package Manager (<https://p3m.dev/client/>) is large. Plus, {rig} released a beta yesterday that manages R packages.

Docker: what a container does

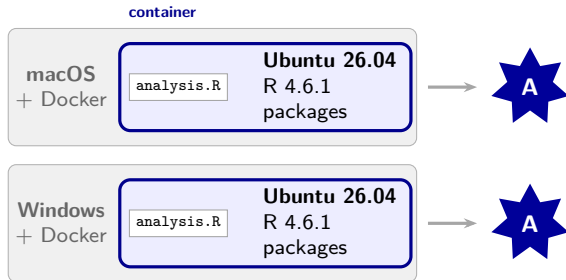
Dockerfile (recipe) → **image** (the environment, frozen) → **container** (the image, running)

Without a container



different setups, (potentially) different outputs

With a container



the container brings its own OS and R: same output

Nix: what it is

A package manager (+ language + build tool) designed so that **installing software gives the same result every time**.

- ▶ From Eelco Dolstra's PhD work at Utrecht (first paper 2004, thesis 2006); used for parts of CERN's LHCb experiment.
- ▶ Environments are built from an explicit, declarative description (`default.nix`).
- ▶ Every package lives in its own folder under `/nix/store`, named by a hash of everything that went into building it.

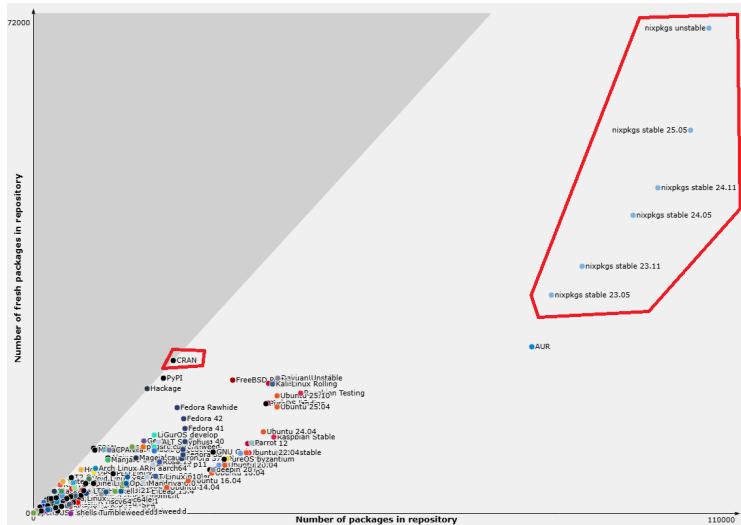
(Burr, Clemencic, & Couturier, 2019; Dolstra, de Jonge, & Visser, 2004; Rodrigues & Baumann, 2026)

Nix: what it is (continued)

- ▶ `nixpkgs`: one repository for R, nearly all of CRAN, Bioconductor, and more. See <https://search.nixos.org/packages>.
- ▶ Binary caches make it fast enough for daily interactive work.
- ▶ Works without root access (e.g., HPC):
<https://github.com/nix-community/nix-user-chroot>. For the people in Flanders, I have tested this (thanks to Balázs Hajgató). Otherwise, inside Apptainer.

(Burr, Clemencic, & Couturier, 2019; Dolstra, de Jonge, & Visser, 2004; Rodrigues & Baumann, 2026)

Nix, the package manager: size



Source: <https://repology.org/repositories/graphs>, figure as reproduced in Rodrigues & Baumann (2026).

Nix: the empirical evidence

Malka, Zacchiroli, & Zimmermann (2025) rebuilt **709,816 packages** from nixpkgs snapshots taken every four months, 2017–2023:

- ▶ **Rebuildable**: 99.7% to 99.9% still build from their description.
- ▶ **Bitwise reproducible**: 69% in 2017, 91% in 2023, rising.
- ▶ **Functionally reproducible**: not measured, but follows for every package that rebuilds. From my understanding, every input is pinned by hash, so a rebuild can fail but cannot succeed with different versions. This is not the same with Docker.

(Malka, Zacchiroli, & Zimmermann, 2025)

Things that are hard to learn, in increasing order of difficulty

- ▶ Making Italians happy with my carbonara
- ▶ Nix, the programming language
- ▶ Belgian politics

Nix: what an expression looks like

```
let
  pkgs = import (fetchTarball
    "https://github.com/rstats-on-nix/nixpkgs/archive/2026-08-31.tar.gz") {};
in
pkgs.mkShell {      # output: a development shell
  buildInputs = [   # what goes inside it
    pkgs.R
    pkgs.rPackages.broom
    pkgs.rPackages.ggplot2
  ];
}
```

(Dolstra, de Jonge, & Visser, 2004; Rodrigues & Baumann, 2026)

{rix}: Nix without writing Nix

One R call generates the environment specification:

```
rix::rix(  
  date = "2026-08-31",           # pin a snapshot date  
  r_pkgs = c("quarto", "broom", "ggplot2"), # CRAN packages  
  system_pkgs = "quarto",       # command-line tools  
  tex_pkgs = c("amsmath", ...),  # LaTeX packages  
  ide = "positron",  
  project_path = "."  
)
```

Writes default.nix. Then, in the terminal:

```
nix-build          # build the environment  
nix-shell --pure   # enter it - R, packages, Quarto, LaTeX all pinned
```

Same default.nix → same environment

Issues encountered thus far

- ▶ **Misdeclared dependencies:** an R package wrongly listed a dependency under Imports when it belonged in Suggests.
- ▶ **Parsing bug:** a dependency written as `R(>= x)` with no space and a package whose name ends in R. Fixed already in the GitHub development version.
- ▶ **macOS:** the description stays valid, but an OS update can break today's build. Recommendation: build inside a dockerfile.
- ▶ **Isolation:** use `nix-shell --pure`, so the shell sees only the declared environment and not software installed on the host.

Illustration: live demo

```
# gen-env.R: the only file written by hand
rix::rix(date = "2026-08-31",
        r_pkgs = c("quarto", "broom", "ggplot2"),
        system_pkgs = "quarto", tex_pkgs = c("amsmath", ...),
        ide = "")
```

<code>Rscript gen-env.R</code>	<code># writes default.nix (+ .Rprofile)</code>
<code>nix-build</code>	<code># build the environment once</code>
<code>nix-shell --pure</code>	<code># enter it</code>
<code>quarto render analysis.qmd</code>	<code># render inside it (HTML + PDF)</code>

- ▶ `default.nix` is plain text: commit it, share it, rebuild it anywhere.
- ▶ Read the documentation for `{rix}`:
<https://docs.ropensci.org/rix/articles/getting-started.html>

Illustration: full study

End to end: simulation → figures → APA manuscript, re-rendered inside the pinned environment:

```
nix-shell --pure  
quarto render Manuscript/article.qmd
```

Paper & materials: <https://doi.org/10.5281/zenodo.21098113>

What is next?

- ▶ Developing {rixcheck}: <https://github.com/felipelfv/rixcheck>

rixcheck

lifecycle

experimental

R-CMD-check.yaml

passing



Continuous reproducibility checks for Nix-pinned R projects.

A project whose computational environment is described by a `default.nix` (for example one generated with [rix](#)) makes a strong promise: the environment can be rebuilt from its description alone, years later, on any Linux machine. **rixcheck turns that promise into a routinely tested fact.** It scaffolds a GitHub Actions workflow (and, optionally, a Dockerfile) that:

1. rebuilds the pinned environment from the committed `default.nix`,
2. re-runs the project's analysis and/or renders its manuscript inside it,
3. optionally compares the freshly produced outputs against the versions committed to the repository.

Step 3 upgrades the check's claim from *"this project is executable"* to *"this project still produces its published numbers, on Linux, from its recipe."*

References I

- Ahrens, W., Demmel, J., & Nguyen, H. D. (2020). Algorithms for efficient reproducible floating point summation. *ACM Transactions on Mathematical Software*, 46(3).
<https://doi.org/10.1145/3389360>
- Baker, D. H., Berg, M., Hansford, K. J., Quinn, B. P. A., Segala, F. G., & Warden-English, E. L. (2024). ReproduceMe: Lessons from a pilot project on computational reproducibility. *Meta-Psychology*, 8, MP.2023.4021. <https://doi.org/10.15626/MP.2023.4021>
- Bouma, A., van Assen, M. A. L. M., van Aert, R. C. M., & Voncken, L. (2026). Reporting practices, open science practices, and trustworthiness of simulation studies in psychology: A questionnaire study. *Advances in Methods and Practices in Psychological Science*.
- Burr, C., Clemencic, M., & Couturier, B. (2019). Software packaging and distribution for LHCb using Nix. *EPJ Web of Conferences*, 214, 05005. <https://doi.org/10.1051/epjconf/201921405005>
- Chan, C., & Schoch, D. (2023). Rang: Reconstructing reproducible R computational environments. *PLOS ONE*, 18(6), e0286761. <https://doi.org/10.1371/journal.pone.0286761>
- Csárdi, G. (2026). *Rig: The R installation manager*. <https://github.com/r-lib/rig>

References II

- Dolstra, E., de Jonge, M., & Visser, E. (2004). Nix: A safe and policy-free system for software deployment. *18th Large Installation System Administration Conference*, 79–92.
- Epskamp, S. (2019). Reproducibility and replicability in a fast-paced methodological world. *Advances in Methods and Practices in Psychological Science*, 2(2), 145–155. <https://doi.org/10.1177/2515245919847421>
- Fontana Vieira, F., Geller, J., & Rodrigues, B. (2026). *felipelfv/Why-risk-it-when-you-can-rix-it: rv4* (Version rv4). Zenodo. <https://doi.org/10.5281/zenodo.21281624>
- Goldberg, D. (1991). What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1), 5–48. <https://doi.org/10.1145/103162.103163>
- Grønneberg, S., Foldnes, N., & Marcoulides, K. M. (2022). Covsim: An R package for simulating non-normal data for structural equation models using copulas. *Journal of Statistical Software*, 102(3), 1–45. <https://doi.org/10.18637/jss.v102.i03>
- Lamb, C., & Zacchiroli, S. (2022). Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software*, 39(2), 62–70. <https://doi.org/10.1109/ms.2021.3073045>

References III

- Landau, W. M. (2021). The targets r package: A dynamic make-like function-oriented pipeline toolkit for reproducibility and high-performance computing. *Journal of Open Source Software*, 6(57), 2959. <https://doi.org/10.21105/joss.02959>
- Levenstein, M. C., & Lyle, J. A. (2018). Data: Sharing is caring. *Advances in Methods and Practices in Psychological Science*, 1(1), 95–103. <https://doi.org/10.1177/2515245918758319>
- Luijken, K., Lohmann, A., Alter, U., Claramunt Gonzalez, J., Clouth, F. J., Fossum, J. L., Hesen, L., Huizing, A. H. J., Ketelaar, J., Montoya, A. K., Nab, L., Nijman, R. C. C., Penning de Vries, B. B. L., Tibbe, T. D., Wang, Y. A., & Groenwold, R. H. H. (2024). Replicability of simulation studies for the investigation of statistical methods: The RepliSims project. *Royal Society Open Science*, 11(1), 231003. <https://doi.org/10.1098/rsos.231003>
- Malka, J., Zacchiroli, S., & Zimmermann, T. (2024). Reproducibility of build environments through space and time. *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*, 97–101. <https://doi.org/10.1145/3639476.3639767>
- Malka, J., Zacchiroli, S., & Zimmermann, T. (2025). Does functional package management enable reproducible builds at scale? yes. *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*, 775–787. <https://doi.org/10.1109/msr66628.2025.00115>

References IV

- Malka, J., Zacchiroli, S., & Zimmermann, T. (2026). Docker does not guarantee reproducibility. *arXiv Preprint arXiv:2601.12811*. <https://arxiv.org/abs/2601.12811>
- Montoya, A. K., & Anderson, S. F. (2026). Repeatability in monte carlo simulation studies. *Advances in Methods and Practices in Psychological Science*.
- Morris, T. P., White, I. R., & Crowther, M. J. (2019). Using simulation studies to evaluate statistical methods. *Statistics in Medicine*, 38(11), 2074–2102. <https://doi.org/10.1002/sim.8086>
- Navarro, D. (2025). *When good pseudorandom numbers go bad*. Notes from a data witch (blog). https://blog.djnavarro.net/posts/2025-05-18_multivariate-normal-sampling-floating-point/
- Niethammer, M., Strobl, C., Boulesteix, A.-L., Fried, E. I., Morris, T. P., Sauer, C., Bonneville, E. F., Aczel, B., Hoogeveen, S., Sarafoglou, A., & Hoffmann, S. (2026). Crowdsourcing methodological comparisons: A many-analysts simulation study to identify conditions under which results in convenience samples generalize to target populations. *Advances in Methods and Practices in Psychological Science*.

References V

- Nosek, B. A., Alter, G., Banks, G. C., Borsboom, D., Bowman, S. D., Breckler, S. J., Buck, S., Chambers, C. D., Chin, G., Christensen, G., Contestabile, M., Dafoe, A., Eich, E., Freese, J., Glennerster, R., Goroff, D., Green, D. P., Hesse, B., Humphreys, M., ... Yarkoni, T. (2015). Promoting an open research culture. *Science*, 348(6242), 1422–1425. <https://doi.org/10.1126/science.aab2374>
- Ottoboni, K., & Stark, P. B. (2018). *Random problems with r*. <https://arxiv.org/abs/1809.06520>
- Pawel, S., Bartoš, F., Siepe, B. S., & Lohmann, A. (2026). Handling missingness, failures, and non-convergence in simulation studies: A review of current practices and recommendations. *The American Statistician*, 80(1), 31–48. <https://doi.org/10.1080/00031305.2025.2540002>
- Pawel, S., Kook, L., & Reeve, K. (2024). Pitfalls and potentials in simulation studies: Questionable research practices in comparative simulation studies allow for spurious claims of superiority of any method. *Biometrical Journal*, 66(1), 2200091. <https://doi.org/10.1002/bimj.202200091>
- Peikert, A., & Brandmaier, A. M. (2021). A reproducible data analysis workflow. *Quantitative and Computational Methods in Behavioral Sciences*, 1, e3763. <https://doi.org/10.5964/qcmb.3763>
- Peikert, A., van Lissa, C. J., & Brandmaier, A. M. (2021). Reproducible research in R: A tutorial on how to do the same thing more than once. *Psych*, 3(4), 836–867. <https://doi.org/10.3390/psych3040053>

References VI

- Rodrigues, B. (2023). *Building reproducible analytical pipelines with r*. <https://raps-with-r.dev>
- Rodrigues, B., & Baumann, P. (2026). *Nix for polyglot, reproducible data science workflows* (Version v0.0.1). Zenodo. <https://doi.org/10.5281/zenodo.18138618>
- Sauer, C., Lange, F. J. D., Thurow, M., Dormuth, I., & Boulesteix, A.-L. (2026). Statistical parametric simulation studies based on real data. *WIREs Computational Statistics*, 18(3). <https://doi.org/10.1002/wics.70074>
- Siepe, B. S., Bartoš, F., Boulesteix, A.-L., Heck, D. W., Peikert, A., Sarafoglou, A., & Pawel, S. (2026). Why, when, and how to (or not to) preregister a simulation study. *Advances in Methods and Practices in Psychological Science*.
- Siepe, B. S., Bartoš, F., Morris, T. P., Boulesteix, A.-L., Heck, D. W., & Pawel, S. (2024). Simulation studies for methodological research in psychology: A standardized template for planning, preregistration, and reporting. *Psychological Methods*. <https://doi.org/10.1037/met0000695>
- Simonsohn, U., & Gruson, H. (2026). *Groundhog: Version-control for CRAN, GitHub, and GitLab packages*. <https://CRAN.R-project.org/package=groundhog>
- Stan Development Team. (2025). *Reproducibility*. <https://mc-stan.org/docs/reference-manual/reproducibility.html>

References VII

- Strobl, C., Chalmers, R. P., & Henninger, M. (2026). SIMP²L: A template for preregistration, piloting documentation, and transparency over the entire life-cycle of a simulation study. *Advances in Methods and Practices in Psychological Science*.
- Ushey, K., & Wickham, H. (2024). *Renv: Project environments*.
<https://CRAN.R-project.org/package=renv>
- Wiebels, K., & Moreau, D. (2021). Leveraging containers for reproducible psychological research. *Advances in Methods and Practices in Psychological Science*, 4(2).
<https://doi.org/10.1177/25152459211017853>
- Ziemann, M., Poulain, P., & Bora, A. (2023). The five pillars of computational reproducibility: Bioinformatics and beyond. *Briefings in Bioinformatics*, 24(6), bbad375.
<https://doi.org/10.1093/bib/bbad375>